

Verifikation verteilter Systeme in Isabelle

Ein Vergleich von Kleppmann und I/O-Automaten

Sepp Stage

TU Chemnitz, Fakultät Betriebssysteme



TECHNISCHE UNIVERSITÄT
CHEMNITZ

1. Einleitung

2. Das Zwei-Phase-Commit-Protokoll

- Übergänge des Koordinators

- Übergänge eines Teilnehmers

- Beispielhafte Abläufe

3. I/O Automaten

- Signatur

- Mathematische Definition

- Ausführung

- Komposition

4. Vergleich

- Modellierung

- Invarianten

- Beweis einer Invarianten

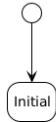
- Beweis der einheitlichen Zustimmung

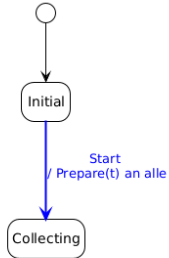
5. Vergleich

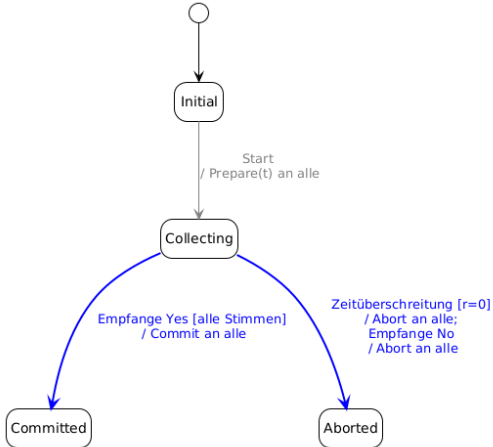
- Kriterien...

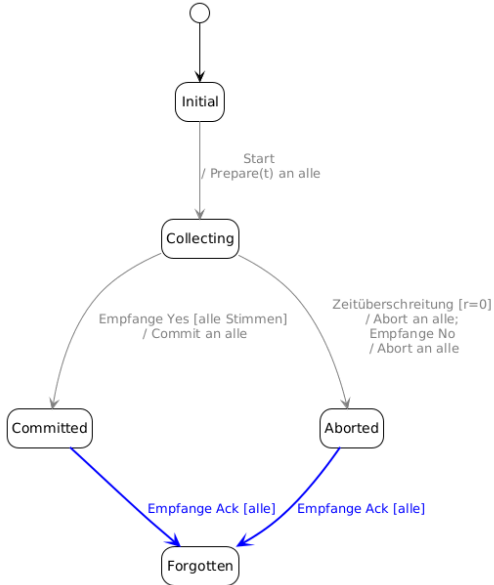
6. Fazit

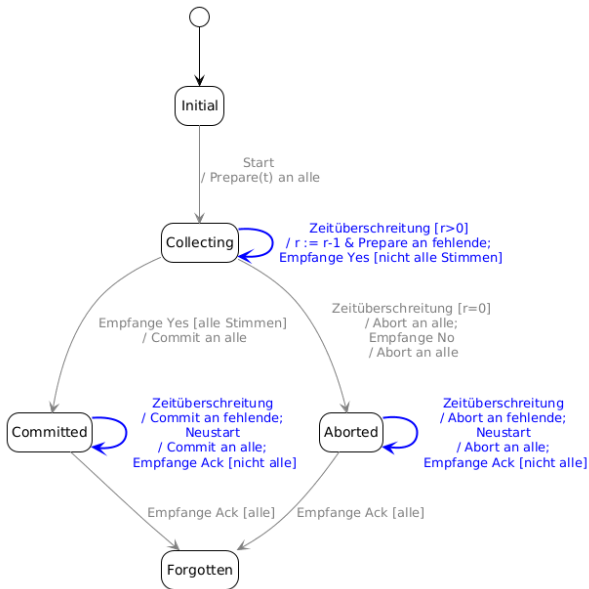
AT&T

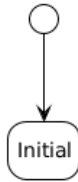


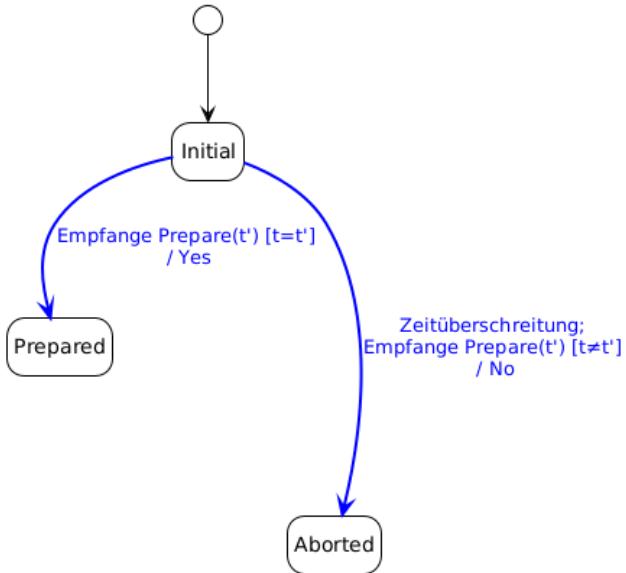


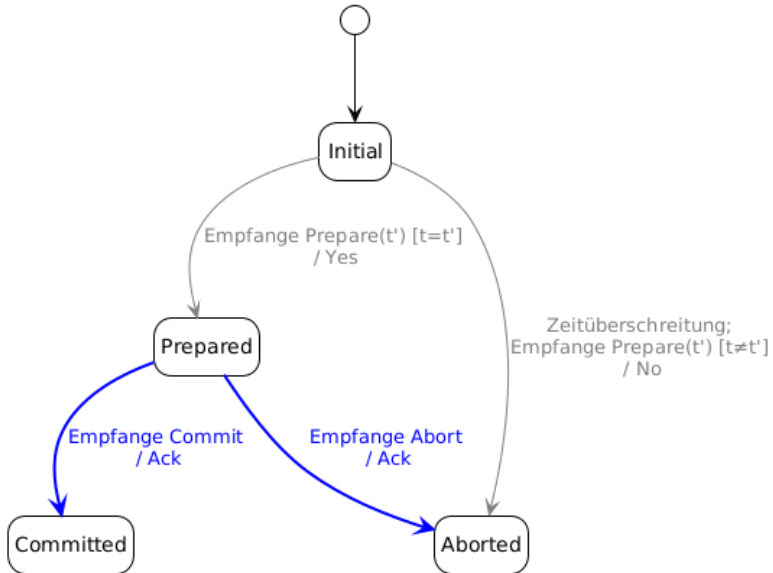


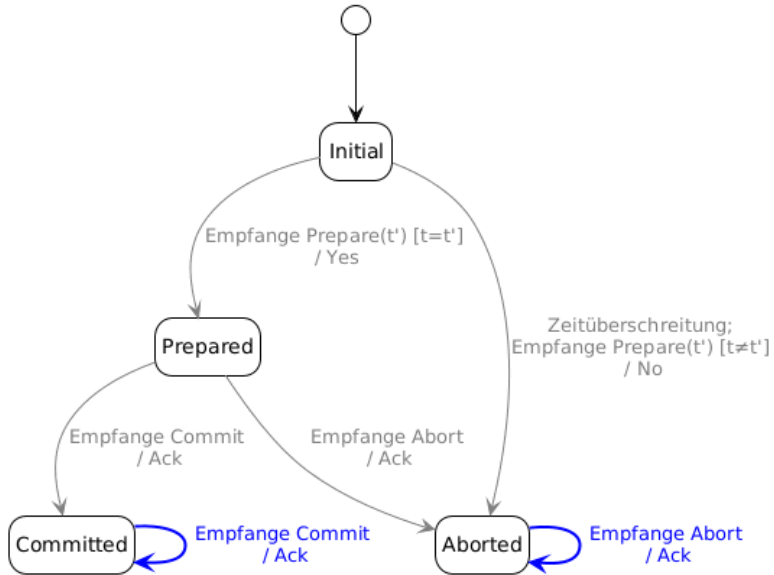


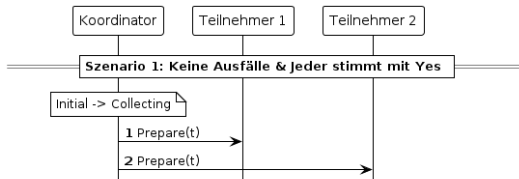


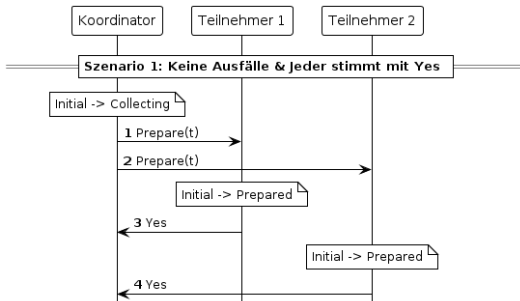


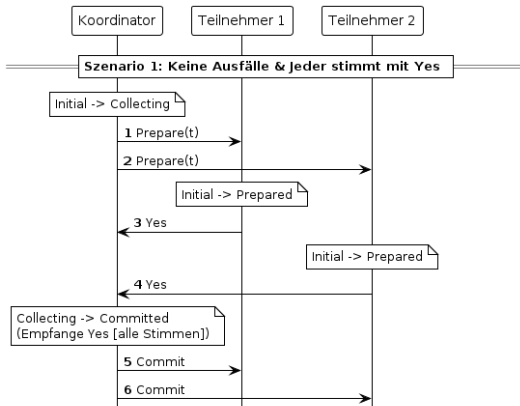


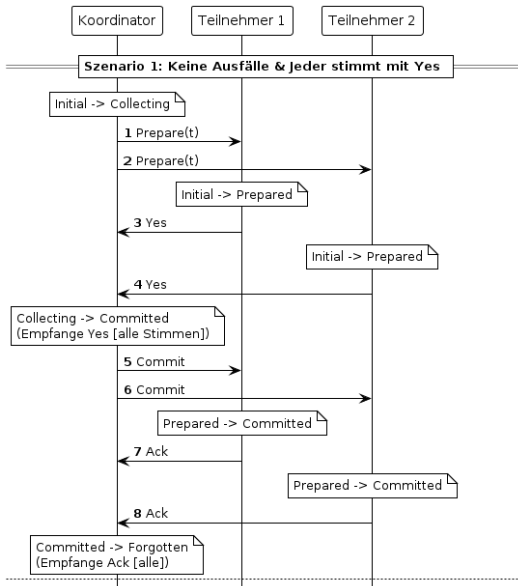


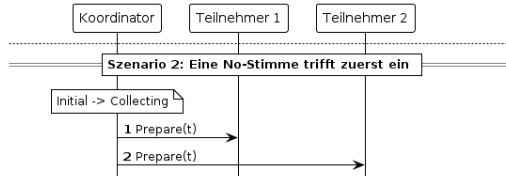


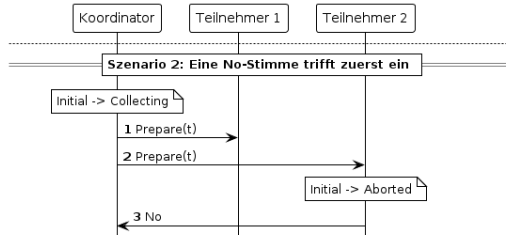


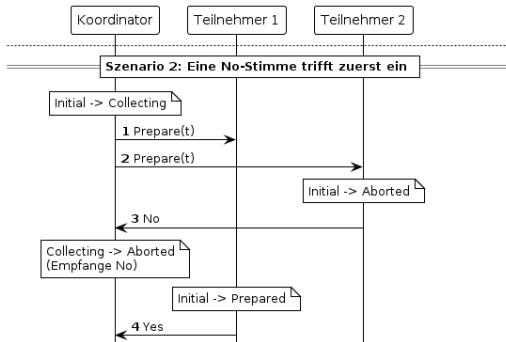


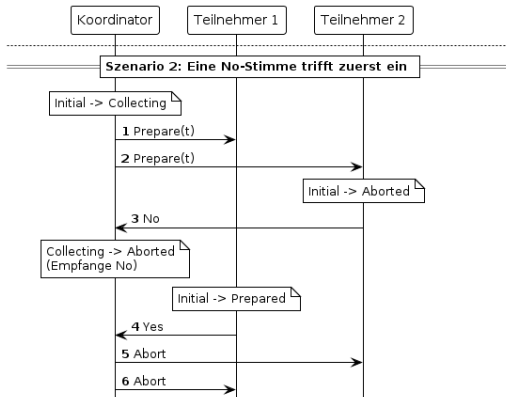


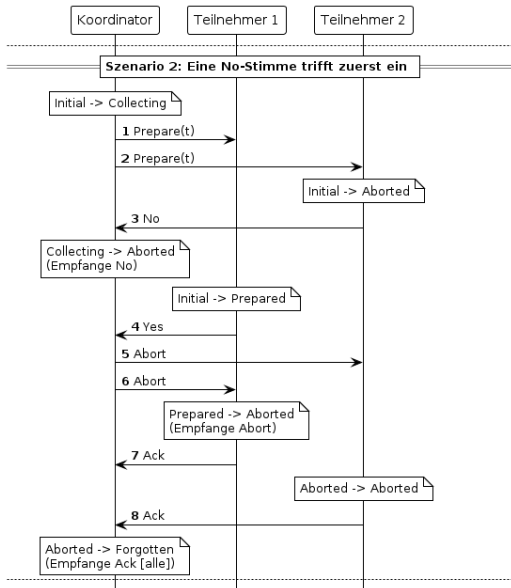


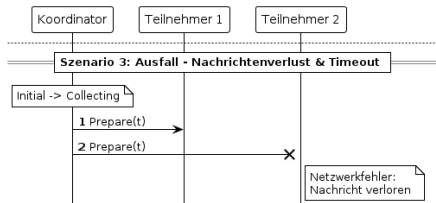


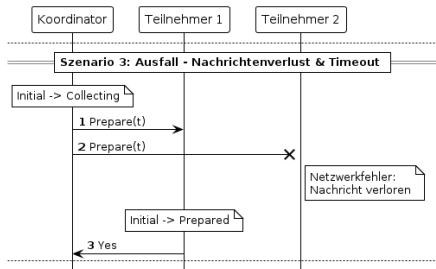


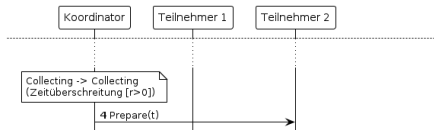
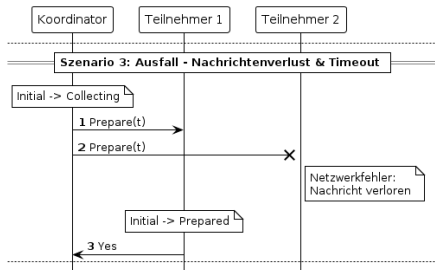


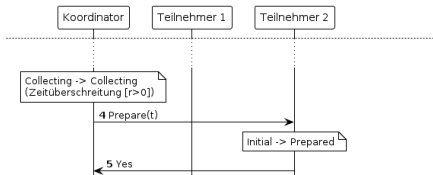
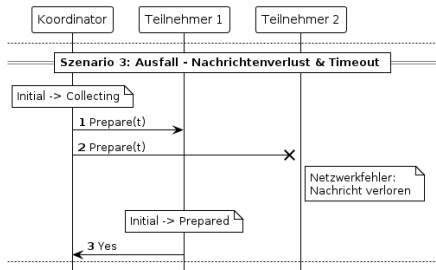


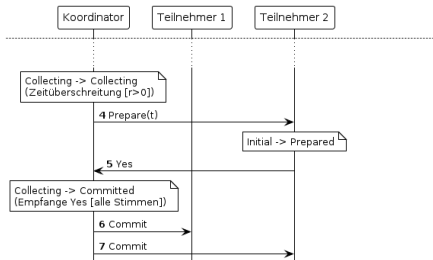
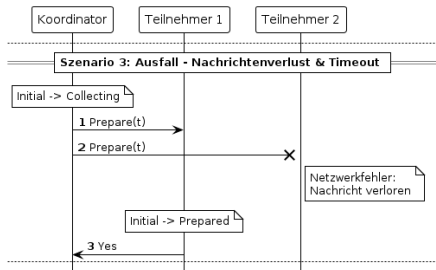


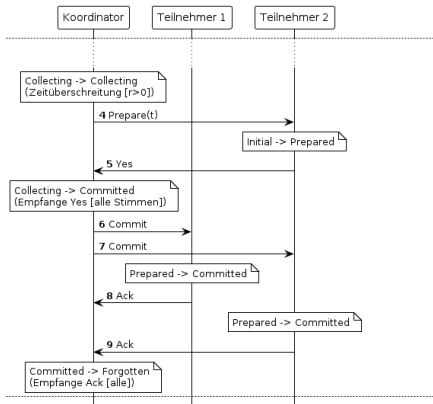
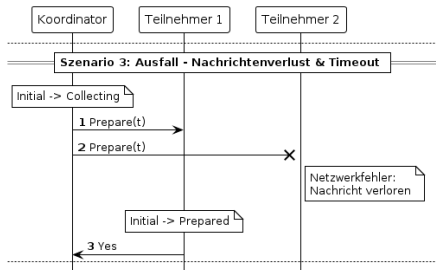


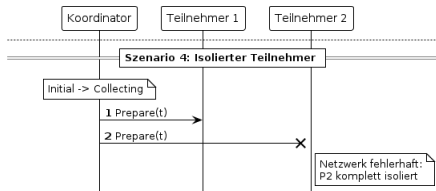


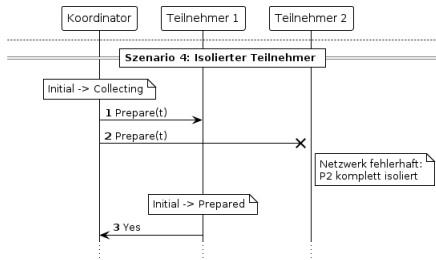


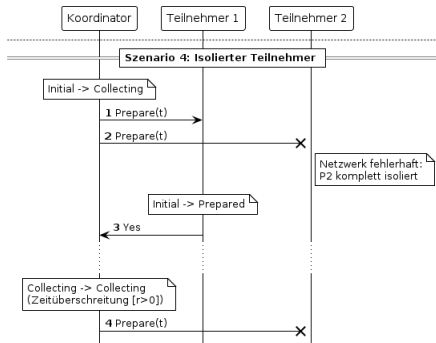


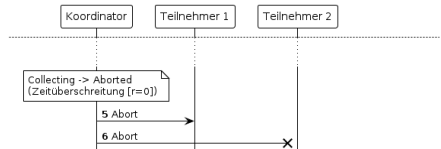
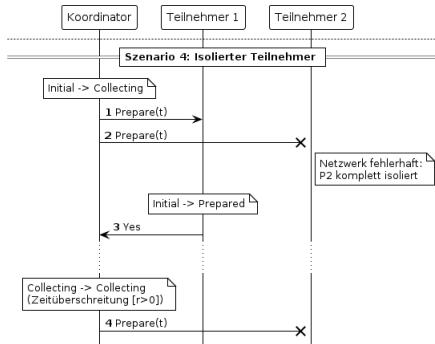


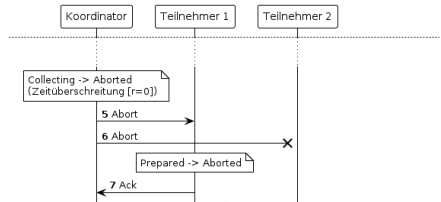
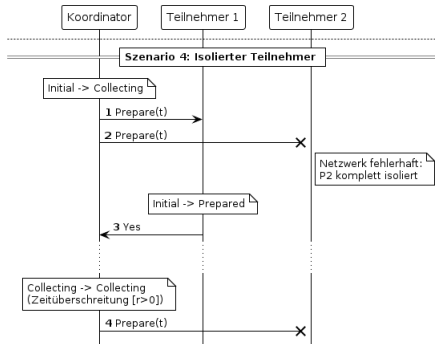


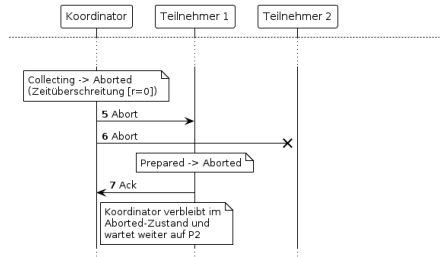
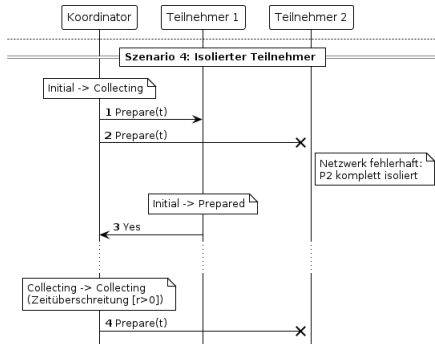


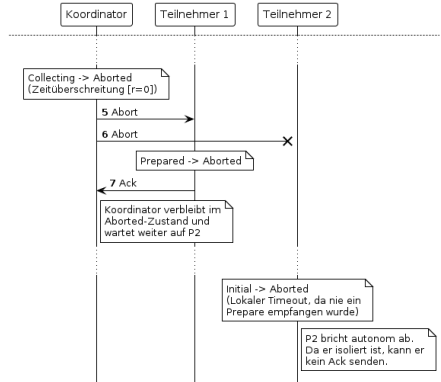
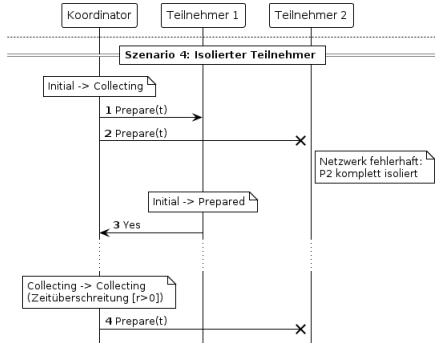












Definition einer Signatur S

- ▶ *Input-Übergänge* $in(S)$
- ▶ *Output-Übergänge* $out(S)$
- ▶ *Internal-Übergänge* $int(S)$

Beispielbild!!!!

Definition eines I/O Automaten A

- ▶ *Signatur* $sig(A)$
- ▶ *Menge an Zuständen* $states(A)$
- ▶ *nichtleere Menge an Startzuständen* $start(A) \subseteq states(A)$
- ▶ *Übergangsrelation*

$$trans(A) \subseteq states(A) \times acts(sig(A)) \times states(A)$$

- ▶ *Aufgabenpartition* $tasks(A)$

Beispielbild????

Ausführung eines I/O Automaten A

- ▶ *Ausführungsfragment* $s_0, \pi_1, s_1, \pi_2, \dots$
- ▶ $\forall i \geq 0. (s_i, \pi_{i+1}, s_{i+1}) \in \text{trans}(A)$
- ▶ *Ausführung falls* $s_0 \in \text{starts}(A)$

Beispielbild????

Komposition zweier I/O Automaten A & B

- ▶ *Automaten müssen kompatibel sein*
- ▶ *Signatures werden Verknüpft*
- ▶ *entstehende Zustände sind Vektoren (s_a, s_b) mit $s_i \in \text{start}(I)$*
- ▶ *A und B dürfen bei gemeinsamen Übergängen gleichzeitig agieren*

OUTPUTS bleiben OUTPUTS für weitere Komposition!

```

5  datatype 't msg = Prepare (transaction: 't) | Yes | No |
   ↳ Commit | Abort | Ack
10  datatype ('proc, 'msg) event
11    = Start
12    | Receive (msg_sender: 'proc) (recv_msg: 'msg)
13    | Timeout
14    | Restart
  
```

```

6  datatype 't msg = Prepare (transaction: 't) | Yes | No |
   ↳ Commit | Abort | Ack
8  datatype ('proc, 'msg) action =
9    Start "'proc"
10   | Send "'proc" "'proc" "'msg"
11   | Receive "'proc" "'proc" "'msg"
12   | Timeout "'proc"
13   | Restart "'proc"
  
```

```

50 definition coord_asig :: "'proc  $\Rightarrow$  ('proc,'t) action
     $\hookrightarrow$  signature" where
51 "coord_asig pid =
52   ({Receive q pid m | q m. True}  $\cup$  {Start pid, Timeout pid,
     $\hookrightarrow$  Restart pid},
53   {Send pid q m | q m. True},
54   {})"

89 definition parts_asig :: "'proc set  $\Rightarrow$  ('proc,'t msg) action
     $\hookrightarrow$  signature" where
90 "parts_asig P =
91   ({Receive q p m | q p m. p  $\in$  P}  $\cup$  {Timeout p | p. p  $\in$  P}  $\cup$ 
     $\hookrightarrow$  {Restart p | p. p  $\in$  P},
92   {Send p q m | p q m. p  $\in$  P},
93   {})"

113 definition chan_asig :: "('proc,'t) action signature" where
114 "chan_asig =
115   ({Send s r m | s r m. True},
116   {Receive s r m | s r m. True},
117   {})"
  
```

```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten

```

```

17 datatype ('t, 'proc) cstate = CInitial (transaction: 't)
   ↪  (all: "'proc set") (retry_count:"nat")
18 | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
19 | CCommitted (all: "'proc set") (ack: "'proc set")
20 | CAborted (all: "'proc set") (ack: "'proc set")
21 | Forgotten
22 datatype ('t, 'proc) coord_state = CState "('t, 'proc) cstate"
   ↪  "('proc × 'proc × 't msg) set"

```

```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:""'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

```

61 datatype 't pstate = PInitial (transaction: 't)
62 | Prepared
63 | PCommitted
64 | PAborted
65 datatype ('t, 'proc) part_state = PState "'t pstate" "('proc
   ↪  × 'proc × 't msg) set"
  
```

```

7  datatype ('t, 'proc) state = Initial (transaction: 't) (all:
   ↪  "'proc set") (retry_count:"nat")
8  | Collecting (transaction: 't) (all: "'proc set")
   ↪  (yes:"'proc set") (retry_count:"nat")
9  | Prepared
10 | Committed (all: "'proc set") (ack: "'proc set")
11 | Aborted (all: "'proc set") (ack: "'proc set")
12 | Forgotten
  
```

```

106 type_synonym ('proc, 't) chan_state = "('proc × 'proc × 't
   ↪  msg) set"
  
```

```

14 fun coordinator_step::
15   "'proc ⇒ ('t, 'proc) state ⇒ ('proc, 't msg) event ⇒
    ⇨ ('t, 'proc) state × ('proc, 't msg) send set" where
  
```

```

34 fun participant_step::
35   "'('t, 'proc) state ⇒ ('proc, 't msg) event ⇒ ('t,
    ⇨ 'proc) state × ('proc, 't msg) send set" where
  
```

```

24 fun coordinator_step::
25   "'proc ⇒ ('t, 'proc) cstate ⇒ ('proc, 't msg) action ⇒
    ⇨ ('t, 'proc) cstate × ('proc × 't msg) set" where
  
```

```

67 fun participant_step::
68   "'proc ⇒ 't pstate ⇒ ('proc, 't msg) action ⇒ 't pstate
    ⇨ × ('proc × 't msg) set" where
  
```

```

16  <coordinator_step pid (Initial t a r) Start = (Collecting t
    ↪  a {pid} r, {Send p (Prepare t) | p. p ∈ a})> |

26  <coordinator_step pid (CInitial t a r) (Start _) =
    ↪  (Collecting t a {pid} r, {(q, (Prepare t)) | q. q ∈ a -
    ↪  {pid}})> |
  
```

```

17  <coordinator_step pid (Collecting t a y r) (Receive sender
    ↪  msg) =
18    (case msg of
19      Yes ⇒(if  $y \cup \{\text{sender}\} = a$  then (Committed a {pid},
        ↪  {Send p (Commit) |  $p. p \in a$ }) else (Collecting t
        ↪  a ( $y \cup \{\text{sender}\}$ ) r, {}))) |
20      No ⇒(Aborted a {pid}, {Send p (Abort) |  $p. p \in a$ }) |
21      _ ⇒(Collecting t a y r, {}))> |
  
```

```

27  <coordinator_step pid (Collecting t a y r) (Receive sender
    ↪  _ msg) =
28    (case msg of
29      Yes ⇒(if  $y \cup \{\text{sender}\} = a$  then (CCommitted a {pid},
        ↪  {(q, Commit) |  $q. q \in a - \{\text{pid}\}$ }) else
        ↪  (Collecting t a ( $y \cup \{\text{sender}\}$ ) r, {}))) |
30      No ⇒(CAborted a {pid}, {(q, Abort) |  $q. q \in a -$ 
        ↪  {pid}}) |
31      _ ⇒(Collecting t a y r, {}))> |
  
```

```

22  <coordinator_step pid (Collecting t a y 0) Timeout =
    ↪ (Aborted a {pid}, {Send p (Abort) | p. p ∈ a})> |
23  <coordinator_step _ (Collecting t a y (Suc r)) Timeout =
    ↪ (Collecting t a y r, {Send p (Prepare t) | p. p ∈ (a -
    ↪ y)})> |
  
```

```

32  <coordinator_step pid (Collecting t a y 0) (Timeout _) =
    ↪ (CAborted a {pid}, {(q, Abort) | q. q ∈ a - {pid}})> |
33  <coordinator_step pid (Collecting t a y (Suc r)) (Timeout
    ↪ _) = (Collecting t a y r, {(q, Prepare t) | q. q ∈ (a -
    ↪ y)})> |
  
```

```

26   <coordinator_step _ (Committed a ack') Timeout = (Committed
      ↪   a ack', {Send p Commit | p. p ∈ (a - ack')})> |
27   <coordinator_step _ (Committed a ack') Restart = (Committed
      ↪   a ack', {Send p Commit | p. p ∈ a})> |

36   <coordinator_step pid (CCommitted a ack') (Timeout _) =
      ↪   (CCommitted a ack', {(q, Commit) | q. q ∈ (a - ack')})>
      ↪   |
37   <coordinator_step pid (CCommitted a ack') (Restart _) =
      ↪   (CCommitted a ack', {(q, Commit) | q. q ∈ a - {pid}})>
      ↪   |
  
```

```

44 definition coord_trans ::
45   "'proc  $\Rightarrow$  (('t,'proc) coord_state  $\times$  ('proc,'t msg) action  $\times$ 
     $\hookrightarrow$  ('t,'proc) coord_state) set" where
46   "coord_trans pid =
47     {(CState s msgs, a, CState s' (msgs  $\cup$  (( $\lambda$ msg. (pid, msg))  $\hookrightarrow$ 
     $\hookrightarrow$  sent)))) | s a s' msgs sent. coordinator_step pid s a =
     $\hookrightarrow$  (s', sent)  $\wedge$  ( $\exists$ s m. a = Start pid  $\vee$  a = Receive s pid m
     $\hookrightarrow$   $\vee$  a = Timeout pid  $\vee$  a = Restart pid)}
48    $\cup$  {(CState s msgs, Send pid rcpt msg, CState s (msgs -
     $\hookrightarrow$  {(pid,rcpt,msg)})) | s msgs rcpt msg. (pid,rcpt,msg)  $\in$ 
     $\hookrightarrow$  msgs}"

56 definition Coordinator :: "'p  $\Rightarrow$  't  $\Rightarrow$  'p set  $\Rightarrow$  nat  $\Rightarrow$ 
     $\hookrightarrow$  (('p,'t msg) action, ('t,'p) coord_state) ioa" where
57   "Coordinator pid t a r = (coord_asig pid, {CState (CInitial t
     $\hookrightarrow$  a r) {}}, coord_trans pid, {}, {})"
  
```

```

95 definition parts_trans :: "'proc set  $\Rightarrow$  (('proc  $\Rightarrow$  ('t,'proc)
 $\hookrightarrow$  part_state)  $\times$  ('proc,'t msg) action  $\times$  ('proc  $\Rightarrow$ 
 $\hookrightarrow$  ('t,'proc) part_state)) set" where
96 "parts_trans P = { (s, a, s') .
97    $\forall p$ . if  $p \in P \wedge ((\exists q m. a = \text{Receive } q \ p \ m) \vee a = \text{Timeout } p$ 
 $\hookrightarrow \vee a = \text{Restart } p \vee (\exists q m. a = \text{Send } p \ q \ m))$ 
98   then  $(s \ p, a, s' \ p) \in \text{parts\_trans } p$ 
99   else  $s' \ p = s \ p \}$ "
100
101 definition Participants :: "'proc set  $\Rightarrow$  ('proc  $\Rightarrow$  't)  $\Rightarrow$ 
 $\hookrightarrow$  (('proc,'t msg) action, 'proc  $\Rightarrow$  ('t,'proc) part_state)
 $\hookrightarrow$  ioa" where
102 "Participants P t = (parts_asig P,  $\{\lambda p. P\text{State } (P\text{Initial } (t$ 
 $\hookrightarrow p)) \{\}\}$ , parts_trans P,  $\{\}$ ,  $\{\}\}$ "

```

```

108 definition chan_trans :: "('proc,'t) chan_state × ('proc,'t
    ↪ msg) action × ('proc,'t) chan_state) set" where
109 "chan_trans =
110   {(M, Send s r m, M ∪ {(s,r,m)}) | M s r m. True}
111   ∪ {(M, Receive s r m, M) | M s r m. (s,r,m) ∈ M}"
112
113 definition Channel :: "('proc,'t msg) action, ('proc,'t)
    ↪ chan_state) ioa" where
114 "Channel = (chan_asig, { {} }, chan_trans, {}, {})"
  
```

```

47 fun tupac_step where
48   <tupac_step proc = (if proc = 0 then coordinator_step proc
    ↪   else participant_step)>
  
```

```

124 definition System where
125 "System t P r = ((Coordinator 0 (t 0) (P ∪ {0}) r || Channel)
    ↪   || Participants P t)"
  
```

"By itself, the I/O automaton model has very little structure, which allows it to be used for modelling many different types of distributed systems. Additional structure must be added to the basic model to enable it to describe particular types of asynchronous systems." Lynch buch s.199

b|b|b|b